SECURITY TESTING REPORT

Web Application Security Assessment using DVWA

Application	Damn Vulnerable Web Application (DVWA)
Testing Areas	SQL Injection, Cross-Site Scripting (XSS), Cross-Site Request Forgery (CSRF)
Testing Environment	Local lab environment
Evidence	Screenshots captured during controlled security testing

This report documents controlled testing of three common web application vulnerabilities in DVWA. Each section records the attack scenario, observed evidence, security impact, and the corresponding mitigation approach.

1. Executive Summary

The assessment focused on SQL Injection (SQLi), Cross-Site Scripting (XSS), and Cross-Site Request Forgery (CSRF). The screenshots demonstrate successful vulnerability testing in the local DVWA lab and the associated defensive controls.

Vulnerability	Observed Test	Mitigation
SQL Injection	Malicious SQL input altered the database query behavior.	Prepared statements / parameterized queries
XSS	Script payload executed in the application context.	Output encoding, input validation, and CSP
CSRF	A crafted request was captured/tested against the password change function.	Anti-CSRF tokens and validation

2. SQL Injection (SQLi)

Attack scenario: The DVWA SQL Injection module was tested with a crafted input designed to change the logic of the backend SQL query. The evidence shows the injected query behavior and the resulting database records.

Impact: SQL Injection can allow unauthorized retrieval or manipulation of database information when user input is concatenated directly into SQL statements.

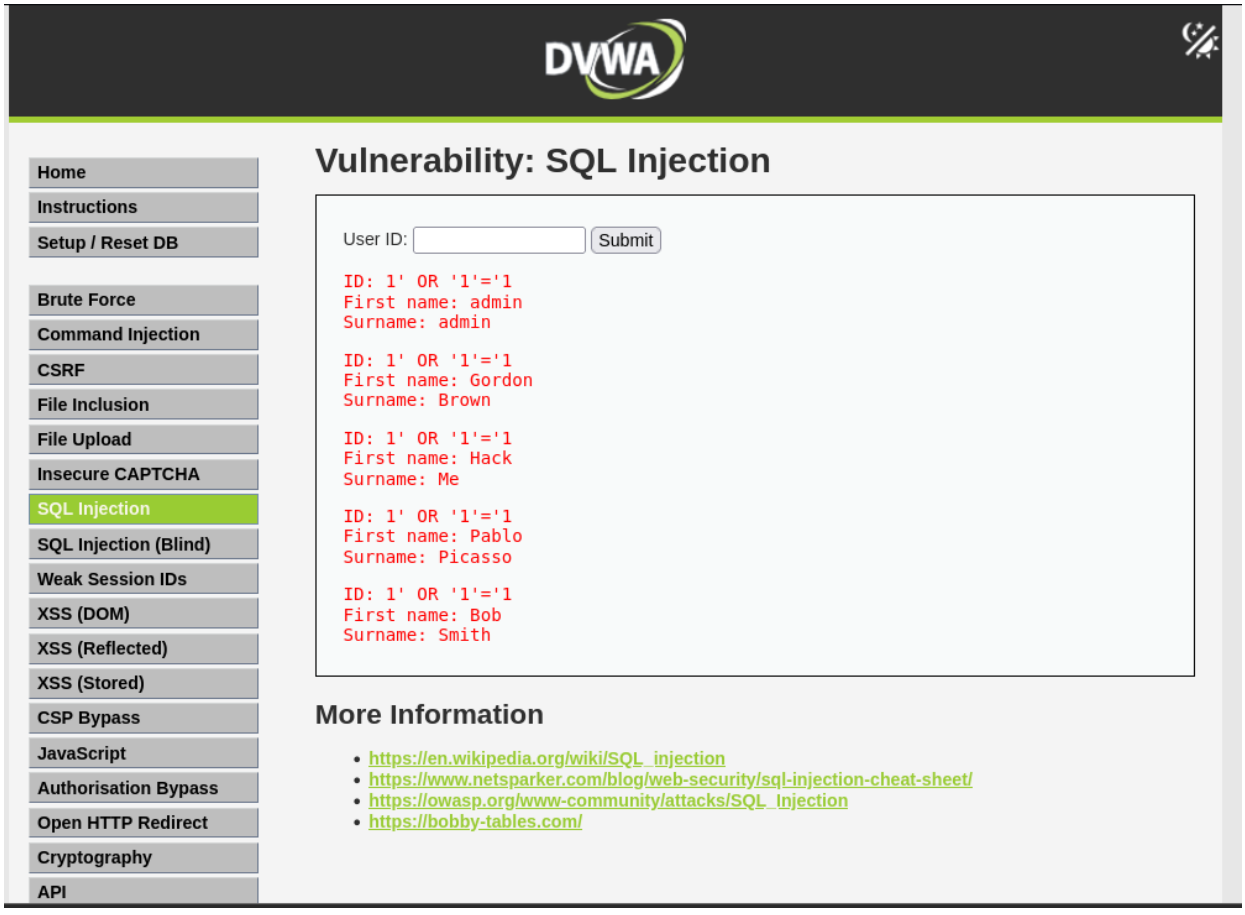


Figure 1. SQL Injection payload and resulting database output in DVWA.

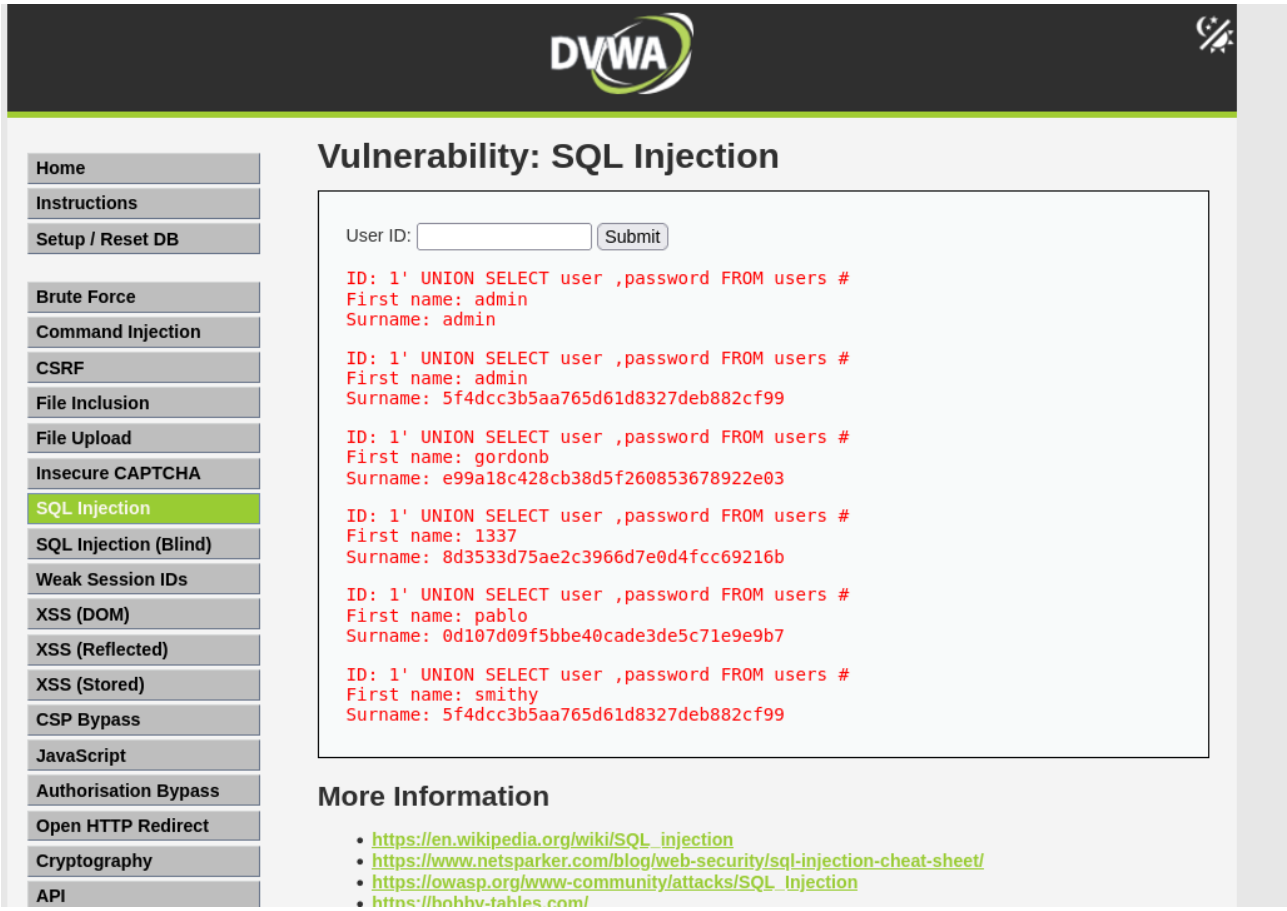


Figure 2. SQL Injection used to retrieve additional database information in the controlled lab.

Mitigation: Use prepared statements / parameterized queries so that user input is treated as data rather than executable SQL syntax. Additional controls include server-side input validation, least-privilege database accounts, and safe error handling.

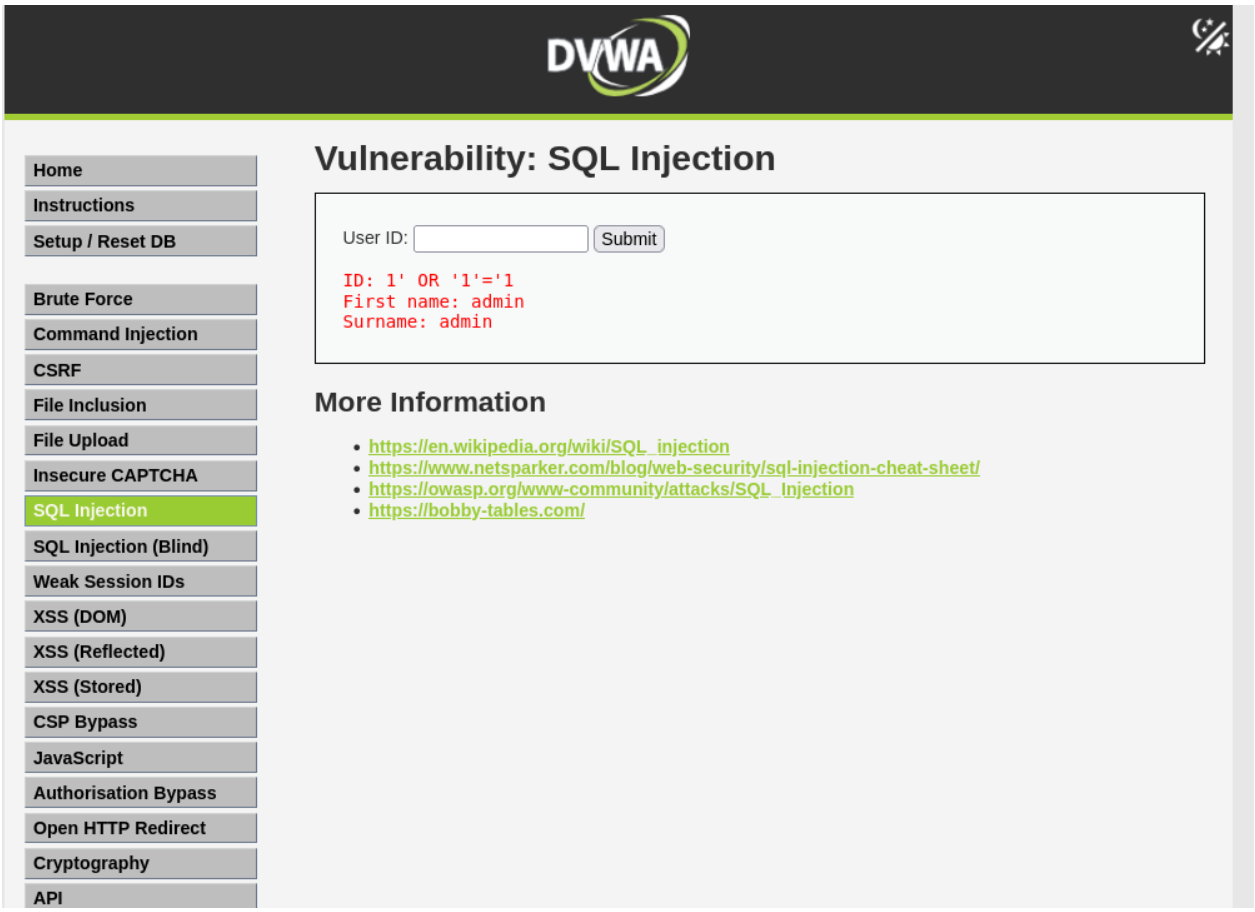


Figure 3. SQL Injection mitigation state showing the parameterized/prepared-statement implementation.

3. Cross-Site Scripting (XSS)

Attack scenario: XSS was tested in DVWA using a JavaScript payload in the XSS functionality. The evidence includes both reflected and stored XSS behavior.

Impact: Successful XSS can execute attacker-controlled script in a victim's browser within the application's origin. Depending on the application, this may affect page content, user actions, or session-related data.

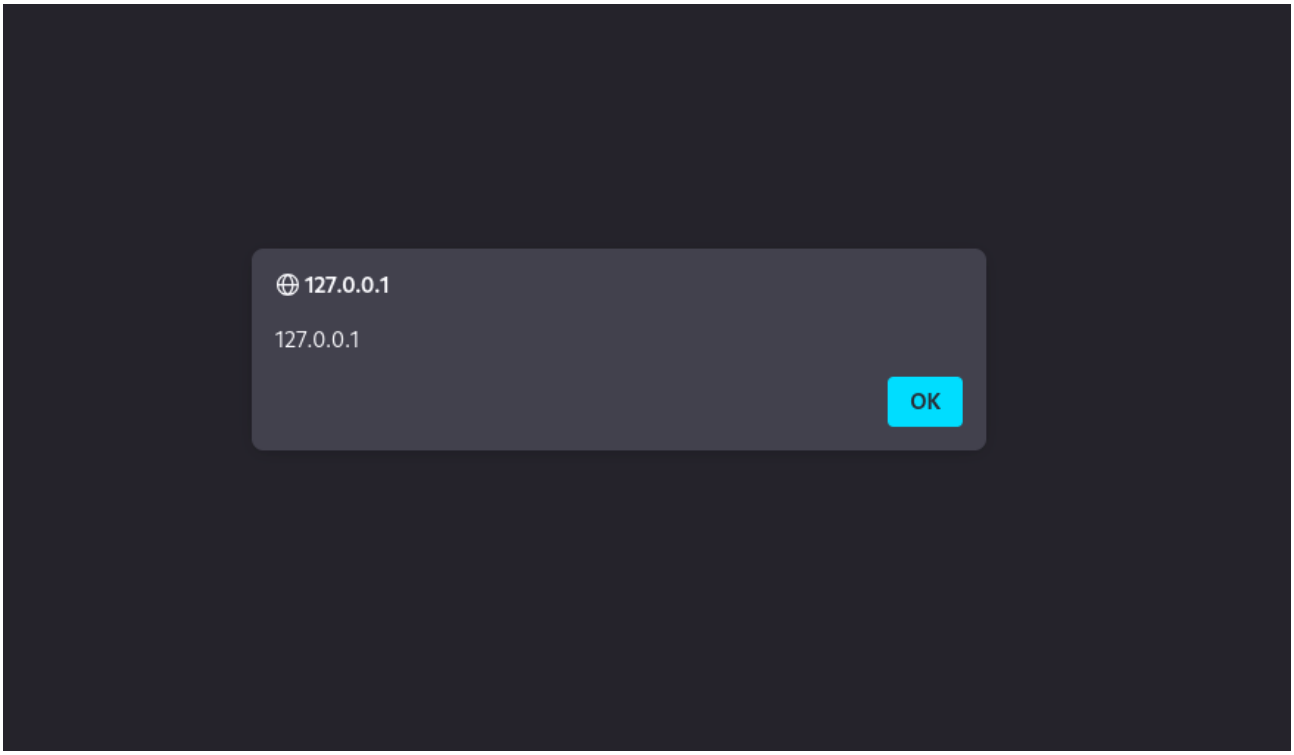


Figure 4. Reflected XSS payload resulting in JavaScript execution.

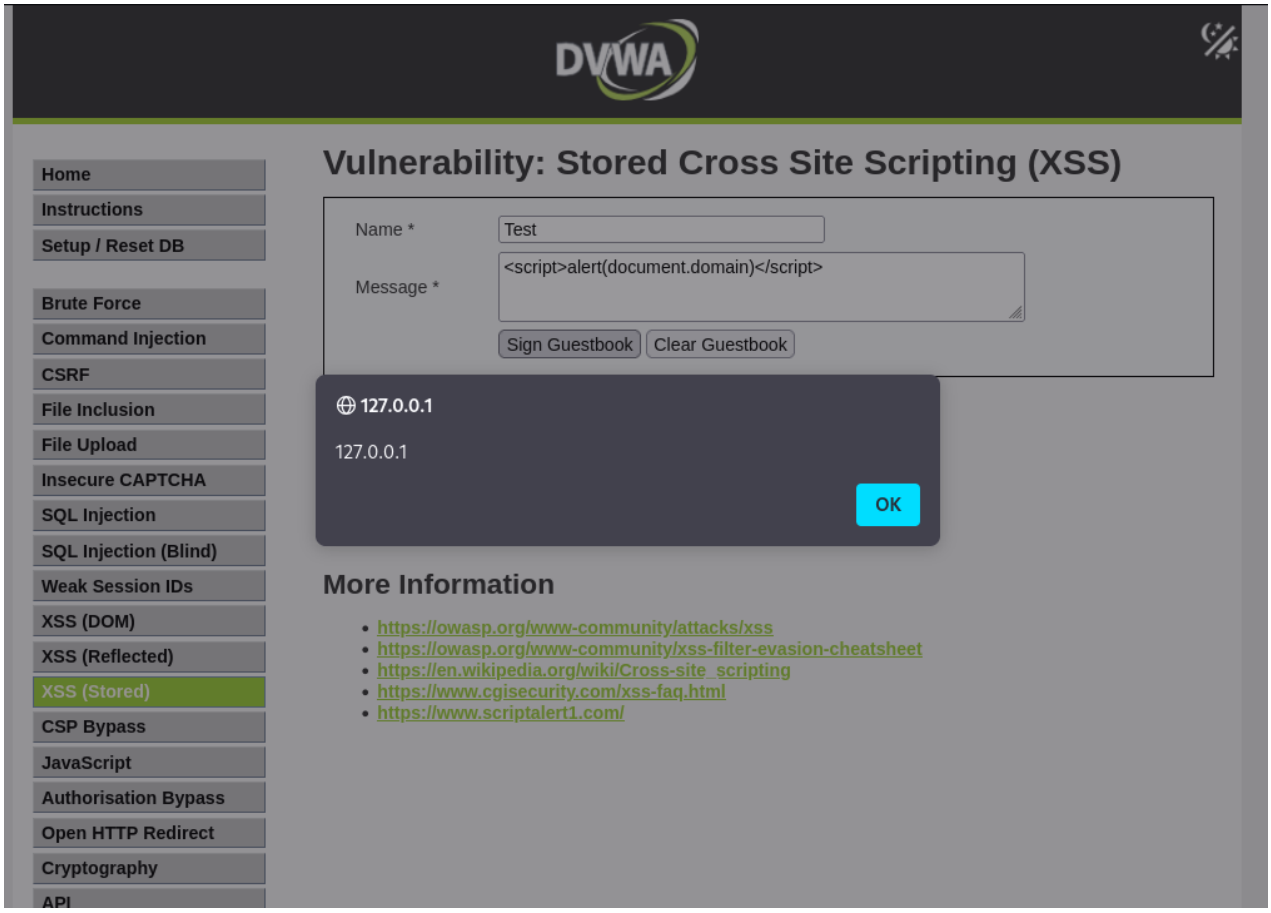


Figure 5. Stored XSS payload executing when the stored content is rendered.

Mitigation: Apply context-aware output encoding, validate input where appropriate, and deploy a restrictive Content Security Policy (CSP). Security-sensitive cookies should also use appropriate flags such as HttpOnly and Secure where applicable.

4. Cross-Site Request Forgery (CSRF)

Attack scenario: The DVWA CSRF function was tested by capturing the password-change request. The request parameters were inspected in the controlled Burp Suite environment to demonstrate how an unauthorized request could be constructed when adequate request validation is absent.

Impact: CSRF can cause an authenticated victim's browser to submit an unwanted state-changing request to a vulnerable application.

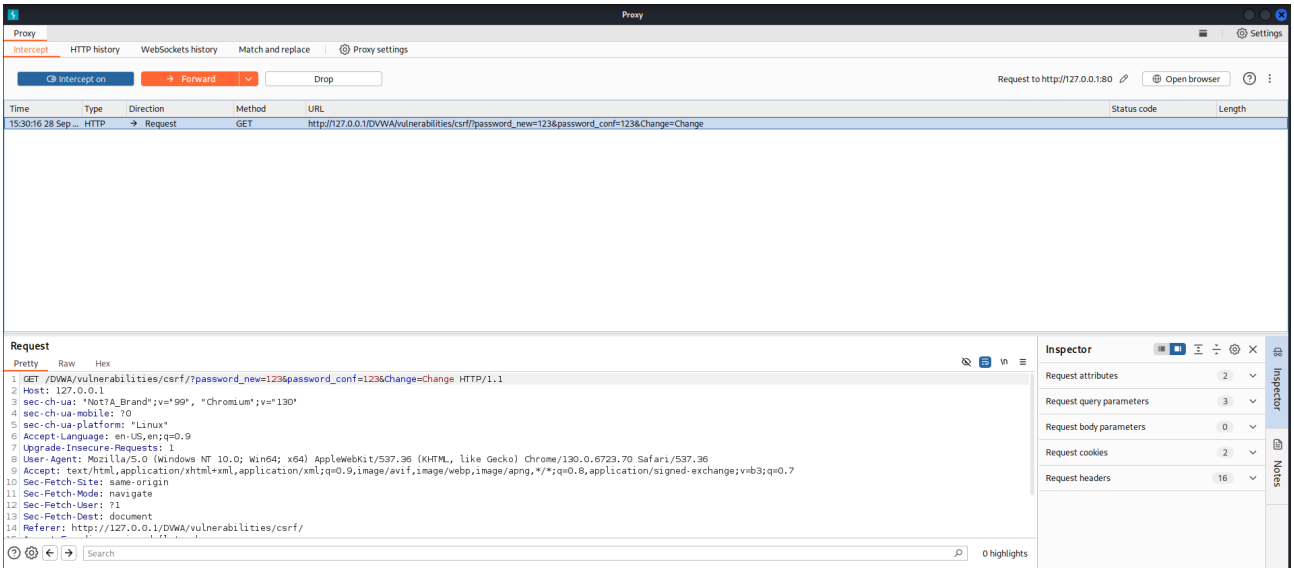


Figure 6. CSRF testing evidence captured in Burp Suite.

Mitigation: Use unpredictable, session-bound anti-CSRF tokens and verify the token on every state-changing request. SameSite cookie settings, origin/referrer validation where appropriate, and re-authentication for sensitive actions provide additional protection.

5. Testing Methodology

1. Reconnaissance: Confirmed the DVWA application and the relevant vulnerability modules in the local lab.

2. Controlled exploitation: Supplied test payloads only to the local DVWA application.

3. Evidence collection: Captured screenshots of successful vulnerability behavior and request/response evidence.

4. Mitigation review: Documented the appropriate defensive control for each vulnerability.

5. Validation: Re-tested the relevant functionality after applying security controls where applicable.

6. Conclusion

The controlled DVWA assessment demonstrated three major web application security weaknesses: SQL Injection, XSS, and CSRF. The report pairs each attack scenario with a corresponding mitigation strategy. Implementing parameterized database queries, context-aware output encoding with CSP, and robust CSRF token validation reduces the risk associated with these vulnerability classes.

Note: All testing documented in this report was performed against the controlled DVWA lab environment.